LLM ENGINEERING

Bonus chapter 1

Bonus: Tokenization

Self-paced reading

Interactive version: https://llm.strueber.dev/#/bonus-01-tokenization

Bethgelab · University of Tübingen · Winter term 2026/27

Text and figures under CC BY 4.0, code under the MIT licence.

Why subwords

1

Why not feed the model raw bytes?

Every text is already a sequence of bytes.

Answer A1

BPE

2

BPE learns an ordered list of merges

- **Byte Pair Encoding** (BPE) (Sennrich et al., 2016)
- Start from base symbols
- Merge the most frequent adjacent pair
- Record each merge's rank
- Stop at a size or frequency target

▼ Training records ranked merges

Byte Pair Encoding (BPE) is a widely used way to learn a subword vocabulary. It is a family of methods rather than a single algorithm (Sennrich et al., 2016). Sennrich-style BPE starts from characters. Byte-level BPE, the subject of the Bytes tab, starts from bytes. Both learn the same thing, an ordered list of merges.

Training starts from a corpus split into base symbols. It counts every pair of adjacent symbols, picks the most frequent pair, merges every occurrence of that pair into one new symbol, and records the merge with its rank. The first merge has rank 1, the second rank 2, and so on. Then it counts again on the merged corpus and repeats. When several pairs share the highest count, an explicit tie rule picks one, so training gives the same result every time. Training stops at a declared target, such as a number of merges, a vocabulary size, or a minimum pair frequency.

Training and encoding answer different questions. Training decides which merges to learn and in which order. Encoding applies the frozen, ranked list to a new text, after the tokenizer's preprocessing. Frequent adjacent sequences often end up as single tokens. A learned piece need not be a word, though, or a meaningful part of one such as a morpheme, because BPE follows frequency alone.

Training learns ranks; encoding replays them

- Ties: the earliest pair wins
- Encoding applies the ranks in order

Toy character-symbol BPE training on one phrase

Corpus: “the lower the better”

Initial alphabet: ␣ b e h l o r t w

Tie rule: earliest pair in the current corpus wins.

Stop: when no adjacent pair occurs more than once.

Training step 0 of 4 · 20 current symbols

Reset | Previous merge | Apply merge

t h e ␣ l o w e r ␣ t h e ␣ b e t t e r

Selected pair: t + h (frequency 2) → th

Learned merge ranks: none yet

▼ Reading the BPE simulation

The widget trains BPE on the phrase "the lower the better", with characters as base symbols (the widget's initial alphabet) and ␣ for the space. Its tie rule is that the earliest pair in the current corpus wins. It stops when no adjacent pair occurs more than once. Each training step highlights the pair that merges and the occurrences it changes, and you can step back or start over.

The widget learns four merges, each from a pair that occurs twice: `t + h → th`, then `th + e → the`, then `the + ␣ → the␣`, and finally `e + r → er`. Each merge removes two symbols from the corpus, which shrinks from 20 symbols to 18, 16, 14 and finally 12. After the fourth merge every remaining pair occurs only once. The stop rule ends training there. Without it, the tie rule would have to pick among many pairs that each occur once, and the choice would mean nothing.

Once training stops, the widget encodes a new string, "the lower", with the frozen ranks. It starts from nine symbols, `[t, h, e, ␣, l, o, w, e, r]`. Rank 1 makes `th`, rank 2 makes `the`, rank 3 joins the following space into `the␣`, and rank 4 makes `er`. The result is `[the␣, l, o, w, er]`, so 9 symbols become 5 tokens. An implementation therefore stores an ordered merge table. A set of frequent strings would not say which merge to apply first.

The simulation is a toy. A production tokenizer uses a different base, splits the text before merging, aims at a target vocabulary size, and has its own stopping rule.

Toy character-symbol BPE training on one phrase

Corpus: “the lower the better”

Initial alphabet: ␣ b e h l o r t w

Tie rule: earliest pair in the current corpus wins.

Stop: when no adjacent pair occurs more than once.

Training step 4 of 4 · 12 current symbols

Reset | Previous merge | Apply merge

the␣ l o w er ␣ the␣ b e t t er

Learned merge ranks: th, the, the␣, er

Stopped: no adjacent pair occurs more than once.

Frozen-rank encoding of “the lower”

the␣ l o w er

9 characters → 9 initial symbols → 5 encoded tokens

After the fourth merge no pair occurs twice, so training stops. The four frozen ranks then encode “the lower” as five tokens: the␣, l, o, w, er.

4

How does BPE encode “other”?

Use the same four learned merges.

Answer A2

Bytes

5

A 256-byte base leaves no input unknown

- Base: all 256 byte values (Radford et al., 2019)
- `Ġ` displays the space byte
- UTF-8: 1 to 4 bytes per character (Yergeau, 2003)
- NFC `é` is bytes `C3 A9`
- Monad keeps 207 of 256 bytes
- SentencePiece: another route (Kudo & Richardson, 2018)

▼ Coverage needs every byte in the vocabulary

Text becomes bytes in two steps. Unicode gives every character a number, its **code point**, written as U+ followed by hexadecimal digits. The letter `A` is U+0041, and `é` is U+00E9. An encoding then turns each code point into bytes. UTF-8 is by far the most common encoding. On the web, 99.1% of the sites whose encoding is known use it (W3Techs, 2026). Three of its properties matter for tokenizers (Yergeau, 2003). It is variable-length, so a code point takes one to four bytes. It is a prefix code. The first byte of a character tells how many bytes follow, and every following byte starts with the bits `10`. So no character's bytes are the beginning of another character's bytes, and a decoder always knows where a character starts. Finally, it is backward compatible with ASCII, the 128-character basic Latin set. Those characters keep their single byte values `00` to `7F`, so every ASCII text is already valid UTF-8. A few byte values, `C0`, `C1` and `F5` to `FF`, never occur in valid UTF-8 at all.

Byte-level BPE starts from bytes instead of characters. GPT-2 is a prominent example. It maps each of the 256 byte values to a printable symbol and learns its merges on top of that base (Radford et al., 2019). The printable symbols are a display convention. The 188 bytes whose Latin-1 reading is a printable character, such as letters, digits and punctuation, keep that character as their symbol. The other 68, among them the space, the newline and the control bytes, are shifted to unused printable characters. In this map the space byte becomes `Ġ` (U+0120), and the newline `Ċ`. The shift exists because GPT-2's merge code breaks on whitespace and control characters (OpenAI, 2019). Its merges file, for instance, writes each merge as two symbols separated by a space, so a symbol that contained a space would be ambiguous. This is why Week 1's trace contained tokens such as "Ġprice". The course tokenizer uses the same convention.

A character outside ASCII takes more than one byte. The letter `é` in its composed form, NFC (Normalization Form C), is the two UTF-8 bytes `C3 A9`. Suppose all 256 byte symbols are in the vocabulary and preprocessing rejects no input. Then every UTF-8 byte sequence has a starting unit, and no input is unknown. Merges are not needed for this. They only change how many tokens encode the bytes.

The condition matters, because a byte-level tokenizer need not keep all 256 symbols. Monad's vocabulary holds 207 of them. Together with its 7,981 merges and 4 special tokens, they make up its 8,192 entries. The missing symbols are most control bytes, the byte values that never occur in valid UTF-8, and the first bytes of a few ranges of characters. One of these ranges is Thaana, the script of Dhivehi, the language of the Maldives. The others are part of the private-use area and the higher Unicode planes. These gaps fit what Monad is for. PleIAs built it as a candidate for the smallest viable language model, with 56 million parameters, strictly monolingual in English, and with a tokenizer trained only on its own training data, SYNTH (PleIAs, 2025). English text practically never contains the missing bytes, so a model built for English loses little by having no rows for them. Some tokenizers have a byte fallback for such cases, a reserved token for every byte value. Llama 2 writes these tokens as `<0xE5>` and so on. Monad has none. For a Thaana letter such as U+0780 (bytes `DE 80`), a private-use character or a control byte, the missing byte becomes `[UNK]`, and only the character's remaining bytes survive. For the course this gap does no harm in practice. On the first 20,000 English documents of the course's training data, the PleIAs SYNTH dataset, Monad produced about 20 million tokens, averaging 4.17 bytes per token, and not one `[UNK]`.

Byte-level BPE is one design among several. SentencePiece, for example, trains directly from raw sentences, without assuming that spaces separate words, and marks the start of a word with the symbol `▁` (Kudo & Richardson, 2018).

Pre-tokenization fixes where merges may happen

- A regex cuts text into chunks (OpenAI, 2019)
- Merges never cross a chunk
- Llama 3: digit runs of 1 to 3 (Meta, 2024)
- Monad: whole digit runs (PleIAs, 2025)

```
"Pay 1000000 now"
chunks: ["Pay", "Ġ1000000", "Ġnow"]
tokens: ["P", "ay", "Ġ100", "0000", "Ġnow"]
```

▼ Chunks first, merges inside them

Before any merge, a byte-level tokenizer cuts the text into chunks with a regular expression (regex). This step is called **pre-tokenization**. GPT-2's pattern makes a chunk from an optional space followed by a run of letters, an optional space followed by a run of digits, an optional space followed by a run of other symbols, or a run of whitespace, and it treats a few English contractions such as "'s" separately (OpenAI, 2019). BPE then merges only inside a chunk, never across a chunk boundary. This is why a word's leading space travels with the word, as in "Ġprice", and why no merge joins the end of one word to the start of the next.

Pre-tokenization is also where a tokenizer decides how to treat numbers. Llama 3's official tokenizer cuts a run of digits into chunks of one to three digits, and the learned merges then decide the tokens inside each chunk (Meta, 2024). Monad's configuration contains a similar one-to-three-digit rule, but stores it as a literal string instead of a regular expression, so the rule never matches (PleIAs, 2025). The pattern that does run is GPT-2's, which keeps a whole digit run together.

The example shows the effect. In "Pay 1000000 now", the seven digits with their leading space form one chunk, `Ġ1000000`, and Monad's merges split it into `Ġ100` and `0000`. A one-to-three-digit rule would have cut the run into `100`, `000` and `0` before any merge.

Byte coverage does not guarantee a round trip

- NFC `é` : `C3 A9`
- NFD `é` : `65 CC 81`
- Normalization can change the code points
- Record every preprocessing policy

▼ **Coverage and reversibility are different questions**

Coverage asks whether every input has a representation. Reversibility asks whether decoding returns exactly the text that came in. The two can come apart.

Unicode can write the same visible character in more than one way. The letter `é` is either one code point, U+00E9, in the composed form NFC with bytes `C3 A9`, or two code points, `e` followed by a combining acute accent, in the decomposed form NFD (Normalization Form D) with bytes `65 CC 81`. A tokenizer with full byte coverage represents both byte sequences. If the pipeline normalizes its input to NFC first, though, an NFD input comes back as NFC after decoding. It looks the same and has different code points. Case conversion, whitespace cleanup and a policy that rejects some inputs break the round trip in the same way. Monad itself applies no normalization, so an NFD `é` is encoded as its three bytes and decodes back exactly. The comparison widget's inputs, by contrast, were normalized to NFC before tokenization.

So when a user reports that a round trip changed their text although every byte is covered, look first at the stages around the tokenizer, such as normalization, cleanup and decoding. When you compare tokenizers, record these preprocessing policies with every result, because they change what the numbers mean.

Vocabulary

8

Special tokens mark structure, not text

```
id 0  [UNK]              no symbol for a byte
id 1  <|begin_of_text|>  start of a sequence (BOS)
id 2  <|end_of_text|>    end of a sequence (EOS)
id 3  [PAD]              filler for short rows
```

- Beginning and end of sequence, BOS and EOS
- The course adds EOS after every document
- Padding fills rows, and masks hide it
- Literal `<|end_of_text|>` in text becomes ID 2
- EOS also ends a turn in Week 12

▼ **Special tokens are vocabulary entries that mark structure**

Not every vocabulary entry stands for a piece of text. **Special tokens** are entries that the tokenizer or the data pipeline inserts to mark structure, and their names are only conventions. Monad has four, with IDs 0 to 3, and they count towards its 8,192 entries. `[UNK]` (ID 0) is the unknown token from the Bytes tab, which stands in for a byte that has no symbol. `<|begin_of_text|>` (ID 1) marks the beginning of a sequence, called BOS. `<|end_of_text|>` (ID 2) marks the end of a sequence, called EOS. In the course's training data each sequence is one document. `[PAD]` (ID 3) fills positions that hold no text.

Monad's tokenizer adds BOS on its own. Encoded with its default settings, Week 1's sentence "The price is 342 euros." becomes `<|begin_of_text|>` followed by the seven tokens of Week 1's trace, which was encoded with special tokens turned off. The tokenizer adds no EOS. The course's training data does the opposite. Every document is encoded without special tokens and followed by one `<|end_of_text|>`, and the documents are concatenated into one long stream. A training window cut from that stream can hold the end of one document, an EOS and the start of the next, so the model learns to predict EOS where a document ends. During generation, a sampled EOS is the usual signal to stop. The course data contains no BOS, so a model trained on it never sees ID 1. All token counts in this chapter are taken with special tokens off, so the tokenizer's default settings give one token more.

Padding is for batches whose sequences differ in length. The shorter rows are filled with `[PAD]` until the batch is a rectangle. A padding token carries no information, so an attention mask keeps real tokens from attending to it and the loss ignores its positions. Week 3 composes these masks. The course's training windows are fixed-length slices of one stream and need no padding. Inputs of different lengths, such as a batch of prompts to generate from, do.

Special tokens are matched in raw text. With the pinned revision, Monad encodes the string `end<|end_of_text|>next` as `<|begin_of_text|> · end · <|end_of_text|> · n · ext`, so the literal characters become the real EOS with ID 2. A web page that happens to contain the string would insert a document boundary where there is none. A serving stack that encodes user text this way lets a user do the same to a deployed model. A pipeline that wants no boundaries from raw text has to remove or escape such strings, or encode raw text without matching special tokens. OpenAI's tiktoken, for example, by default raises an error when a text contains the string of a special token (OpenAI, 2025).

Chat models build on the same idea. A chat template formats a conversation as text with role markers, and many tokenizers add special tokens for the markers. Llama 3, for example, has `<|start_header_id|>` for a role header and `<|eot_id|>` for the end of a turn (Meta, 2024). The course's template in Week 12 introduces no new special tokens. It ends every turn with EOS, so the same token that separates documents in pre-training also ends a turn in a conversation.

Every vocabulary row costs d_model parameters per table

$$\text{parameters per table} = V \cdot d_{\text{model}}$$

- Input embedding: one table
- Untied output head: a second table
- Course: 8,192 × 512 = 4,194,304

▼ Vocabulary tables and hardware padding

The vocabulary appears in the model twice, at its input and at its output. At the input, the embedding table turns tokens into vectors. Think of each token as a one-hot vector of length `V`, all zeros except for a one at the token's ID. Multiplying it by the table, a `V × d_model` matrix, selects one row, and an implementation simply looks that row up. The rows are learned during training, so they become useful input representations for the transformer blocks. At the output, the model's last hidden state has to be turned back into scores over the vocabulary. A linear layer, the **language-model head** or output head, maps it to one logit per vocabulary entry, and a softmax turns the logits into next-token probabilities. To save parameters, many small models tie the head to the embedding table, so the head reuses the table's parameters, transposed, as its weight matrix (Press & Wolf, 2017). Week 2 builds both in its slides "An embedding table turns IDs into vectors" and "Tying shares one table for input and output".

Each vocabulary entry therefore has one row in the input embedding table. Each row holds `d_model` numbers, where `d_model` is the model's hidden width, the length of every embedding vector. A vocabulary of size `V` costs `V · d_model` parameters per table. The output head has the same shape. If it is untied, it is a second table of the same size. If it is tied, it reuses the input table.

With the course values, `V = 8,192` and `d_model = 512`, one table has 8,192 × 512 = 4,194,304 parameters, and two untied tables have 8,388,608. The tables' share of the whole model depends on its depth, its width and the tying, so the table size alone does not tell you that the embedding dominates. Compute the share for your configuration.

The logical vocabulary is the set of IDs the tokenizer can produce. Some implementations pad the matrix to a larger dimension, for example a multiple that suits a GPU kernel. That padding is a separate storage choice. Whether it speeds up a kernel depends on the hardware, the data type and the kernel, so benchmark it on the target setup instead of changing the learned vocabulary to fit a round multiple (Hugging Face, 2026).

10

How many parameters are added?

The vocabulary grows from 8,192 to 32,768 entries at d_model = 512.

Answer A3

More rows shorten sequences but thin the training signal

Dimension	Effect	Measure
Model tables	`V · d_model` per table	Parameters and memory
Sequence length	Larger V may shorten a corpus	Token count, effective context
Piece frequency	Signal spread over more pieces	Frequency and fragmentation
Decision	No size best everywhere	Target-language and domain samples

- Compare on the target mixture (Penedo et al., 2025)
- Opus 4.7: new tokenizer, 1.0–1.35× tokens (Anthropic, 2026)

▼ What a larger vocabulary buys and costs

A larger vocabulary can hold longer pieces, so the same corpus may need fewer tokens. That means shorter sequences and more text per context window. But every new row has to be trained, and a fixed corpus now spreads its training signal across more piece types, so rare pieces see few examples. Quality on rare pieces and across languages therefore need not improve steadily as `V` grows.

Read the table one row at a time. Each row says what changes, how, and how to measure it. The tables grow by `V · d_model` each, which costs parameters and memory. Sequence length may fall, which you measure as token count and use of the effective context, the amount of text that a context window of fixed length holds. Piece frequency spreads out, which you measure as frequency and fragmentation. No vocabulary size is best everywhere, so the decision rests on samples of the target languages and domains.

Compare candidates on the target data mixture itself, by sequence length, piece frequency, and behaviour per language and domain (Penedo et al., 2025). Then measure throughput, effective context and downstream quality.

A tokenizer change in a deployed model shows the token-count side of this trade-off from the user's side. When Anthropic released Claude Opus 4.7 in April 2026, it gave the model an updated tokenizer that, in Anthropic's words, improves how the model processes text. The stated price is that the same input can map to more tokens, roughly 1.0 to 1.35 times as many depending on the content type (Anthropic, 2026). Anthropic does not say how the vocabulary changed, and it kept the per-token prices of the previous model. API use is billed and limited in tokens, so the same prompt can cost more and fill more of the context window than under the previous tokenizer.

Compare tokenizers on one text, not languages on different texts

- `cl100k_base`: OpenAI's GPT-4 tokenizer
- Monad: up to 2.5× `cl100k_base`'s tokens
- Three sentences, not translations
- Monad: Japanese and Bengali stay bytes

How to read the rows

These three sentences are not translations. Compare the tokenizers within one sentence, not one language against another.

Pick a sentence, then read each tokenizer's row: its pieces, their token IDs and the token count.

A piece that is not a complete UTF-8 character appears as hexadecimal bytes in the cl100k_base row, as a byte token such as <0xE5> in the Llama 2 row, and as GPT-2 byte symbols such as å in the Monad row.

The dot (·) stands for a leading space or word boundary, written Ġ or ▁ by the tokenizers themselves. A space inside a piece appears as ‗.

Language: English | Japanese | Bengali

“The transformer architecture revolutionized natural language processing.”

cl100k_base — 9 tokens

The	␣transformer	␣architecture	␣revolution	ized	␣natural	␣language	␣processing	.
791	43678	18112	14110	1534	5933	4221	8863	13

Llama 2 — 10 tokens

·The	·transform	er	·architecture	·revolution	ized	·natural	·language	·processing	.
450	4327	261	11258	19479	1891	5613	4086	9068	29889

Monad — 10 tokens

The	·transform	er	·architecture	·revolution	ized	·natural	·language	·processing	.
307	2914	214	4260	4806	896	1824	2035	2230	17

▼ **The same text can cost very different token counts**

The widget runs three tokenizers on three sentences, one English, one Japanese and one Bengali. The tokenizers are `cl100k_base`, the OpenAI tokenizer of GPT-4 and GPT-3.5 Turbo (OpenAI, 2025), Llama 2's tokenizer (Meta, 2023), and Monad. The length of a token sequence depends strongly on the tokenizer and on the kind of text. The English sentence takes 9 or 10 tokens under all three. The Japanese sentence takes 26 tokens under `cl100k_base`, 27 under Llama 2 and 66 under Monad. The Bengali sentence takes 93, 104 and 217. On the Japanese and Bengali sentences, Monad needs more than twice as many tokens as the other two, up to about 2.5 times as many (66 against 26).

Token counts are costs. Training and inference cost grow with the number of tokens processed, so text that a tokenizer cuts into many short pieces is more expensive to train on and to serve. A context window of fixed length holds less of it, and the model has to connect information across more positions, which makes long contexts harder. Petrov et al. measured this across languages and found that the same text, translated into different languages, can differ up to 15-fold in token count (Petrov et al., 2023). So if your model must handle a particular domain or language well, for example a low-resource language with little training text, you need a tokenizer whose merges cover it. Otherwise training and inference on that text cost much more than on English.

Quality probably suffers too, though the widget does not measure it. An embedding row learns only from the positions that select it. When a script falls apart into single bytes, the same few byte tokens appear inside many unrelated characters and words. Each of their rows then has to serve many meanings at once, and the model has to assemble every character from several positions before it can work with the word.

Read the counts within one sentence. The sentences say different things, so the widget compares different tokenizers on the same sentence. A count for the Japanese sentence and a count for the English one measure different content. They cannot show that a tokenizer handles one language more efficiently than another. That claim needs the same content in both languages, called parallel text, and a large enough sample, as in Petrov et al.'s study.

Within one sentence the rows also show how Monad falls back to bytes. For the Japanese and Bengali sentences its token count equals the byte count, 66 and 217, so every token is a single byte. The widget shows each of these bytes as its GPT-2 display symbol, so `å` or `¤` in Monad's row is one byte, not a Latin letter. No Monad vocabulary entry, whole or partial, contains the multi-byte sequences of these two scripts, so no merge applies. The English sentence, by contrast, goes from 72 bytes to 10 tokens. The other rows show partial characters in their own ways, `cl100k_base` as hexadecimal pieces and Llama 2 as byte-fallback tokens such as `<0xE5>`.

How to read the rows

These three sentences are not translations. Compare the tokenizers within one sentence, not one language against another.

Pick a sentence, then read each tokenizer's row: its pieces, their token IDs and the token count.

A piece that is not a complete UTF-8 character appears as hexadecimal bytes in the cl100k_base row, as a byte token such as <0xE5> in the Llama 2 row, and as GPT-2 byte symbols such as å in the Monad row.

The dot (·) stands for a leading space or word boundary, written Ġ or ▁ by the tokenizers themselves. A space inside a piece appears as ▁.

Language: English | Japanese | Bengali

“大型言語モデルは自然言語処理を変革しました。”

cl100k_base — 26 tokens

大	型	言	e8aa	9e	e383	a2	デ	ル	は	自	然	言	e8aa	9e	e587
27384	25287	78244	45918	252	2845	95	68408	33710	15682	37026	61994	78244	45918	252	12870

a6	理	を	e5a4	89	e99d	a9	し	ました	。
99	22649	30512	8192	231	21688	102	15024	79721	1811

Llama 2 — 27 tokens

·	大	型	言	語	モ	デ	ル	は	自	然	言	語	<0xE5>	<0x87>
29871	30257	30883	31243	30968	30761	30597	30258	30449	30688	31516	31243	30968	232	138

<0xA6>	理	を	変	<0xE9>	<0x9D>	<0xA9>	し	ま	し	た	。
169	30687	30396	31786	236	160	172	30326	30441	30326	30366	30267

Monad — 66 tokens

å	¤	§	å	٦·	ĭ	è	¨	Ģ	è	ª	٦·	ã	ĥ	¢	ã	ĥ	ĩ	ã	ĥ	«	ã	ģ	¯
162	101	104	162	207	188	165	105	177	165	107	207	160	180	99	160	180	184	160	180	108	160	178	111

è	ĩ	ª	ç	Ħ	¶	è	¨	Ģ	è	ª	٦·	å	ĩ	¦	ç	𝕀	Ĩ	ã	Ĥ	Ĵ	å	¤	ī
165	184	107	164	181	118	165	105	177	165	107	207	162	184	103	164	193	183	160	179	195	162	101	186

é	٦·	©	ã	ģ	Ĺ	ã	ģ	¾	ã	ģ	Ĺ	ã	ģ	Ł	ã	Ģ	Ĥ
166	206	106	160	178	200	160	178	126	160	178	200	160	178	208	160	177	179

The Japanese sentence: 26 tokens under cl100k_base, 27 under Llama 2 and 66 under Monad, where every token is a single byte.

Numbers

13

Three tokenizers split 342792 three ways

- Arithmetic reuses place value and carries
- Merges follow frequency, not arithmetic
- Irregular pieces invite plausible, wrong answers
- `_` marks a word boundary

cl100k_base

[342] [792]

Llama 2

[_] [3] [4] [2] [7] [9] [2]

Monad

[34] [27] [92]

▼ Token boundaries and decimal structure

Arithmetic reuses regular structure. Each digit's position stands for a power of ten, carries move from right to left, and the same procedure works for every value. Whether a model can see that structure depends on where the token boundaries fall.

Plain BPE ignores that structure. It merges digit strings by frequency, so common strings such as years or round numbers become tokens of their own, and other numbers split into pieces of varying length. Two numbers that differ in one digit can be cut in different places, and the same digit can sit at a different position inside its token from one number to the next. To learn addition over such pieces, a model would have to learn how every piece combines with every other piece and every operator, so each combination has far fewer training examples than the digits alone would. A model trained only to predict the next token can still learn what a plausible result looks like, such as its length and its likely first digits. Such a model can then produce answers that look right but are wrong.

Tokenizers therefore often treat digits as a special case. Some split every number into single digits, as Llama 2's row on this slide shows. Others cut digit runs into chunks of at most three, as Llama 3 does (see the Bytes tab). Either rule cuts by a fixed pattern that ignores which digit strings are frequent, so the model meets the same small set of digit tokens wherever a number appears. Where the cuts fall relative to place value still matters, as the evidence at the end of this tab shows. A second route keeps the arithmetic out of the token sequence. The model

calls a tool, such as a calculator or a short program it writes and runs, and uses its result. Models learn such tool use in post-training, the stage after pre-training that Weeks 12 and 13 cover.

The three rows show the same six digits, 342792, under three tokenizers. `cl100k_base`, an OpenAI tokenizer, produces `[342][792]`, two groups of three digits. Llama 2 produces `[_][3][4][2][7][9][2]`, one token per digit after a `_`. The `_` marks a word boundary in Llama 2's SentencePiece tokenizer (Meta, 2023). It is not a beginning-of-sequence (BOS) token. Monad produces `[34][27][92]`, three pairs. Every row represents the same input, but the rows group the digits differently relative to decimal positions and carries. BPE learns its merges from text frequency, not from arithmetic rules, so chunks such as `[34][27][92]` need not follow place value or carries.

The token IDs are 17590 and 24763 for `cl100k_base`; 29871, 29941, 29946, 29906, 29955, 29929 and 29906 for Llama 2; and 3754, 3374 and 3626 for Monad. They are indices into a lookup table, not numbers with a magnitude.

14

Where does Monad cut 1,342,792.05?

Its pre-tokenizer keeps digit runs whole.
Can the chunks alone tell you
whether 342 stays one token?

Answer A4

Boundaries change the arithmetic a model must learn

- Alignment and direction changed accuracy (Singh & Strouse, 2024)
- Test the formats you serve
- Signs, decimals, separators, lengths

▼ What the arithmetic evidence shows

Singh and Strouse tested arithmetic in GPT models under different number formats. In the models they studied, how token boundaries aligned with the digits, and whether digits were grouped from the left or from the right, materially changed arithmetic accuracy (Singh & Strouse, 2024). Token boundaries are therefore part of the learning problem. The evidence names no digit policy that is best for every model. It also leaves open whether one-digit tokens generalize better in general and whether multi-digit tokens push a model toward memorizing. Llama 3's one-to-three-digit chunks are one policy among several.

If your system handles numbers, test the tokenizer on them. Build a regression set from the formats and lengths the system will serve, with signed values, decimals, thousands separators, and every length it will see. Check how each candidate tokenizer splits them, and measure the task itself on them.

Measuring and choosing

16

Fertility and PCW count how often words split

- Fertility: tokens per segmented word
- Proportion of continued words (PCW): share split (Rust et al., 2021)
- Name the corpus and word segmenter
- No universal pass or fail thresholds

▼ Two diagnostics and what to report with them

Two diagnostics describe how often a tokenizer splits words. Both need a named corpus and a named word segmenter, the rule that decides what counts as a word. **Fertility** is the number of subword tokens divided by the number of segmented words. The **proportion of continued words (PCW)** is the share of segmented words that are split across more than one token (Rust et al., 2021). Neither has a universal threshold that marks a tokenizer as good or bad.

A small illustrative example shows both. Take `tokenization matters` with a whitespace segmenter. The text is plain ASCII in NFC, with no special tokens added. It has 2 words and, counting the space, 20 bytes and 20 code points. A candidate A that splits it into `[token, ization, matter, s]` has 4 tokens, fertility 4/2 = 2.0, PCW 2/2 = 1.0 because both words are split, and 4/20 = 0.20 tokens per byte and per code point. A candidate B that keeps both words whole, `[tokenization, matters]`, has fertility 1.0, PCW 0, and 0.10 tokens per byte and per code point. B's lower values describe less fragmentation on this phrase. On their own, they do not show that B is the better tokenizer.

A report on tokenizer candidates names, for each one, the corpus, the word segmenter, the normalization and the special-token policy. It gives fertility and PCW where word segmentation is meaningful, and tokens per byte and per code point or grapheme. It adds sequence length, throughput and use of the effective context, and finally downstream loss or task quality. The diagnostics describe a tokenizer on a controlled sample. The target tasks decide whether the differences matter.

17

What are the fertility and PCW?

Monad encodes “tokenization matters”
as to · ken · ization · Ġmatters.

Answer A5

Each denominator answers a different question

Japanese sentence, `cl100k_base`	Value
Tokens	26
UTF-8 bytes	66
Code points	22
Tokens per byte	0.394
Tokens per code point	1.182

- ASCII hides the difference
- Word segmentation depends on the language

▼ Why no single denominator wins

Fertility divides by words. Other diagnostics divide by bytes, by Unicode code points (the numbered characters of the Unicode standard), or by graphemes (what a reader perceives as one character). In ASCII text, bytes and code points are equal, so an English example cannot show that the choice matters.

The Japanese sentence of the comparison widget can. It has 22 code points and 66 UTF-8 bytes, because each of its characters takes three bytes. `cl100k_base` encodes it in 26 tokens: 26/66 = 0.394 tokens per byte, but 26/22 = 1.182 tokens per code point. Per byte, the tokenizer looks compact. Per code point, it needs more than one token. Both numbers are correct, and they answer different questions. Word segmentation depends on the language as well. Japanese does not separate words with spaces, so fertility needs a segmenter built for it.

Choose the denominator that fits the decision. Per byte fits a budget counted in bytes, such as storage or the size of the raw input. Per code point or grapheme fits the length of the text as a reader sees it. A report that gives only one denominator hides what the others would show.

Reuse a tokenizer unless measurements justify training one

- Check data, model, system, outcome
- Training one costs compatibility and validation (Dagan et al., 2024)
- Libraries: `AutoTokenizer`, tiktoken, Gigatoken

Situation	Starting point	Entries
Very small model, English only	Monad	8,192
Medium-sized model	Llama 2	32,000
Medium-sized model	`cl100k_base`	~100k
Many languages, larger tables	`o200k_base`	~200k
Many languages, larger tables	Gemma 4	~262k

▼ **Four questions before changing a tokenizer**

A tokenizer choice affects the data, the model, the system and the results, so four questions cover it. For the data, does the tokenizer represent the target languages, scripts, domains and number formats? For the model, what do its vocabulary tables and sequence lengths cost at the intended width and context? For the system, what throughput and effective context does it give on the actual hardware? For the outcome, what changes in downstream loss and task quality?

In practice you load an existing tokenizer through a library. Hugging Face's `transformers` library loads the tokenizer that belongs to a model repository with `AutoTokenizer.from_pretrained` (Hugging Face, 2026), and the course uses it as a thin wrapper around Monad. OpenAI's tiktoken provides OpenAI's encodings, such as `cl100k_base` and `o200k_base`, the tokenizer of GPT-4o (OpenAI, 2025). Gigatoken is a newer library that reimplements common byte-pair tokenizers for very fast encoding of large corpora. Through a compatibility mode it can stand in for Hugging Face's `tokenizers` library and for tiktoken. Its author reports encoding speeds of gigabytes per second on a many-core server, about a thousand times faster than `tokenizers` (Rød, 2026).

Which tokenizer to start from depends mostly on the model's size and its languages. The table gives our suggestions, each to be checked with the four questions. A very small model trained on English only can use a small vocabulary such as Monad's 8,192 entries, which keeps its tables cheap. A medium-sized model can use Llama 2's 32,000-entry tokenizer or `cl100k_base` with about 100,000 entries. A model that must cover many languages and scripts needs room for merges in all of them, so it uses a larger vocabulary such as `o200k_base`, with about 200,000 entries, or Gemma 4's, with about 262,000 (Gemma Team, 2026), and pays for it with larger embedding tables.

Start from a suitable existing tokenizer. Training a new one, or adapting an existing one, costs more than the training run. The embedding and output tables change, models and data prepared for the old tokenizer no longer match, and everything has to be validated again. Train or adapt only when measured gains justify those costs (Dagan et al., 2024). A shorter corpus is a reason to measure further. If, for illustration, a domain tokenizer shortens your target corpus by 12%, also measure throughput, effective context and downstream quality, and keep the tokenizer you have if those do not improve.

20

So why not feed the model raw bytes?

Answer A6

21

Six results on tokenization

1. Subwords trade vocabulary size for length
2. BPE learns ranks; encoding replays them
3. Check byte coverage, chunks and normalization
4. Vocabulary rows cost parameters, shorten sequences
5. Number boundaries change the arithmetic to learn
6. Measure on target data before choosing

▼ The chapter in six results

Subwords trade vocabulary size for sequence length. Whole words leave unseen words unknown, and raw bytes make sequences long. BPE training learns a ranked list of merges, and encoding replays that list in rank order on new text. A byte-level base leaves no input unknown only if all 256 byte symbols are in the vocabulary. Pre-tokenization decides where merges can happen, and normalization can still change a text on its round trip. Every vocabulary row costs `d_model` parameters per table and can buy shorter sequences, at the price of a thinner training signal per row. Number boundaries change the structure a model has to learn for arithmetic, so test the formats a system serves. Fertility, PCW and per-byte or per-code-point measures describe a tokenizer on a named sample. Target tasks and runtime measurements decide whether to keep a tokenizer or to train a new one.

Reflection questions

No submission. Bring short answers to the Week 2 tutorial.

1. Monad's one-to-three-digit rule is stored as a literal string, so it never runs. Suppose you turn it into a working regular expression but keep Monad's merges and the model trained with them. Does Week 1's sentence "The price is 342 euros." still decode to the same text, how do its tokens change, and what does the model now see that it never saw in training?
2. You are building the tokenizer for a small model that reads English and German product reviews, with prices such as "1.299,00 €" and words such as "Größe". Choose a byte policy, a rule for digits and a vocabulary size, and name the measurements, on which text, that would make you change each choice.
3. Claude Opus 4.7's new tokenizer maps the same input to up to 1.35 times as many tokens. Using what this chapter shows about vocabulary rows, piece frequency and number boundaries, give two reasons a lab might accept more tokens per text, and say what you would measure before making such a switch yourself.

References

23

BPE, Byte and Number Tokenization Sources

- Kudo, T., & Richardson, J. (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. *EMNLP System Demonstrations*.
- Meta. (2024). Llama 3 tokenizer implementation. Software source.
- OpenAI. (2019). GPT-2 byte-pair encoder (src/encoder.py). Software source.
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI technical report.
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*.
- Singh, A. K., & Strouse, D. J. (2024). Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs. *arXiv:2402.14903*.
- W3Techs. (2026). Usage statistics of character encodings for websites. Web survey, accessed 2026-10-19.
- Yergeau, F. (2003). UTF-8, a transformation format of ISO 10646. RFC 3629, Internet Engineering Task Force.

24

Evaluation, Model and Tokenizer Sources

- Anthropic. (2026). Introducing Claude Opus 4.7. Company announcement.
- Dagan, G., Synnaeve, G., & Rozière, B. (2024). Getting the most out of your tokenizer for pre-training and domain adaptation. *ICML*.
- Gemma Team. (2026). Gemma 4 Technical Report. *arXiv:2607.02770*.
- Hugging Face. (2026). Auto Classes: AutoTokenizer. Transformers documentation.
- Hugging Face. (2026). Models: resize_token_embeddings. Transformers documentation.
- Meta. (2023). Llama 2 tokenizer. Model repository.
- OpenAI. (2025). tiktoken. Software repository.
- Penedo, G., Kydlíček, H., Sabolčec, V., et al. (2025). FineWeb2: One Pipeline to Scale Them All: Adapting Pre-Training Data Processing to Every Language. *COLM*.
- Petrov, A., La Malfa, E., Torr, P. H. S., & Bibi, A. (2023). Language Model Tokenizers Introduce Unfairness Between Languages. *NeurIPS*.
- PleIAs. (2025). Monad tokenizer. Model repository.
- Press, O., & Wolf, L. (2017). Using the Output Embedding to Improve Language Models. *EACL*.
- Rød, M. (2026). Gigatoken. Software repository.
- Rust, P., Pfeiffer, J., Vulić, I., Ruder, S., & Gurevych, I. (2021). How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models. *ACL-IJCNLP*.

The comparison widget uses tiktoken (OpenAI, 2025), Llama 2 (Meta, 2023) and Monad (PleIAs, 2025).

Image credits

- Bénard, R. (1769). Compositors setting type at the case. *Encyclopédie, Recueil de planches, vol. VII: Imprimerie en lettres*, Pl. I. Paris Musées, Musée Carnavalet. CC0.

Answers

A1 · Words, bytes and subwords trade vocabulary for length

```
"The transformer architecture revolutionized natural language processing."
```

Unit	Vocabulary	Unknown inputs	Positions
Words	One row per word type	Unseen words	8
Bytes	256	None	72
Monad subwords	8,192	A few characters	10

- Bytes: one position per byte
- Subwords: fixed vocabulary, shorter sequences

▼ Why language models read subwords

Every text a computer stores is already a sequence of bytes. So why does a language model not read those bytes directly? The answer decides how long the model's sequences are, how large its embedding tables become, and what structure in a text the model can see. Training, attention, optimization and scaling all operate on the token sequence that comes out.

Week 1 traced the path from text to model input. A tokenizer cuts the text into pieces, called tokens, and replaces each piece with an integer ID. Each ID selects one row of the embedding table, and that row is the vector the model works with. When the model generates text, the tokenizer maps the generated IDs back to text. This reverse step is called decoding.

The table compares three choices of unit on one sentence, the English sentence of the comparison widget in the Vocabulary tab. The trace below cuts it in all three ways.

```
"The transformer architecture revolutionized natural language processing."
words: ["The", "transformer", "architecture", "revolutionized",
        "natural", "language", "processing", "."]            → 8 positions
bytes: [54, 68, 65, 20, 74, 72, 61, 6E, 73, 66, 6F, 72, …]   → 72 positions
Monad: ["The", "Ġtransform", "er", "Ġarchitecture", "Ġrevolution",
        "ized", "Ġnatural", "Ġlanguage", "Ġprocessing", "."]  → 10 positions
IDs:   [307, 2914, 214, 4260, 4806, 896, 1824, 2035, 2230, 17]
```

The positions column of the table counts these pieces. In UTF-8, the standard byte encoding of Unicode text, the sentence has 72 bytes, written here in hexadecimal. Each position is one row lookup in the embedding table and one step of the model's sequence.

Whole words give the shortest sequence, with seven words and the final period in 8 positions. But the vocabulary needs one row for every word type, and a word that did not occur in training has no row. It

becomes an unknown token, and every unseen word looks the same to the model. Rare and unseen words are the problem that led Sennrich et al. to subword units (Sennrich et al., 2016).

Raw bytes fix the vocabulary at 256 entries, and no input is ever unknown, because every text is a byte sequence. The price is length. Every byte takes one position, so this sentence needs 72. Week 1 showed that more positions mean more attention work. A context window of fixed length also holds less text.

Subwords are the middle ground. The course tokenizer, Monad, has 8,192 vocabulary entries and encodes the sentence in 10 tokens. Frequent words such as "language" are one token each. Rarer words split into pieces, "transformer" into "transform" and "er", and "revolutionized" into "revolution" and "ized". The vocabulary stays fixed, and no word is left without a representation. As in Week 1's trace, a leading "Ġ" marks a token that includes the space in front of it. The last row of the table carries one qualification. Monad's vocabulary lacks some byte symbols, so a few rare characters and control bytes come out as the unknown token `[UNK]`. The Bytes tab explains which bytes are missing and why.

How a tokenizer cuts text therefore sets four things. It sets the sequence length, the size of the vocabulary tables, how well each language and domain is represented, and whether regular structure, such as the digits of a number, stays visible to the model.

This chapter is required reading. It has no submission, and Week 2 builds only on Week 1. It ends with three reflection questions, which the first tutorial, in Week 2, discusses, so prepare a short answer to each.

Back to the question

A2 · Encoding applies ranks in order

- `[o, the, r]`: 5 symbols, 3 tokens
- Rank 2 consumes the `e`
- `e + r` never fires

▼ Encoding “other”, rank by rank

Encoding `other` starts from five symbols, `[o, t, h, e, r]`, and applies the four ranks in order. Rank 1, `t + h`, gives `[o, th, e, r]`. Rank 2, `th + e`, gives `[o, the, r]`. Rank 3 needs `the` followed by a space, and `other` has none, so it does not apply. Rank 4 needs an `e` directly before an `r`, but the `e` is now part of `the`. The result is `[o, the, r]`, three tokens.

The order decides the result. Applied with `e + r` before `th + e`, the same four merges would give `[o, th, er]`. Encoding replays the order in which training learned the merges.

Back to the question

A3 · Quadrupling V adds 12.6M parameters per table

- 24,576 new rows × 512
- Tied head: +12,582,912
- Untied head: +25,165,824

▼ The arithmetic of a larger vocabulary

Going from 8,192 to 32,768 entries adds 32,768 − 8,192 = 24,576 rows. At `d_model = 512` each row has 512 parameters, so one table grows by 24,576 × 512 = 12,582,912 parameters. With a tied output head that is the whole increase. With an untied head both tables grow, by 25,165,824 parameters in total. Whether the change pays off depends on how much shorter the sequences become on the target data and on how downstream quality changes.

Back to the question

A4 · Monad splits 342 differently in each context

```
"1,342,792.05"
chunks: ["1", ",", "342", ",", "792", ".", "05"]
tokens: ["1", ",", "34", "2", ",", "7", "92", ".", "05"]
```

- Cuts at every comma and point
- Chunks alone: no; merges need measuring
- `342` here: `34 · 2`
- In `342792`: `34 · 27 …`
- After a space: `Ġ3 · 42`

▼ Chunks are predictable; merges inside them are not

You can predict the chunks from the pre-tokenizer alone. Commas and the decimal point are punctuation, so each is a chunk of its own, and each digit run between them stays whole: `1`, `,`, `342`, `,`, `792`, `.`, `05`. Which merges apply inside a chunk depends on Monad's learned ranks, so that part cannot be worked out by hand. The measured result is `34 · 2` for `342` and `7 · 92` for `792`.

The same three digits come out three ways in three contexts. Between commas, `342` becomes `34 · 2`. At the start of the run `342792`, the digits become `34 · 27 …`, with the `2` joined to the following `7`. After a space, as in Week 1's sentence "The price is 342 euros.", they become `Ġ3 · 42`. The model therefore meets the same hundreds, tens and units digits in different tokens, with different neighbours, depending on how the number is written.

Back to the question

A5 · Fertility 2 and PCW 0.5 for one short phrase

- 4 tokens, 2 words
- Fertility: 4 / 2 = 2
- PCW: 1 / 2 = 0.5

<details>
<summary>▼ Counting splits on a real segmentation</summary>

With a whitespace segmenter the phrase has 2 words, and Monad's segmentation has 4 tokens, so fertility is 4/2 = 2. Only "tokenization" is split, into `to`, `ken` and `ization`. The word "matters" is a single token with its leading space. So PCW is 1/2 = 0.5, and the phrase costs 4/20 = 0.20 tokens per byte. Compared with candidate A in the fertility example, Monad has the same fertility but a lower PCW, because A splits both words and Monad splits one.

</details>

Back to the question

A6 · Raw bytes cost length; subwords buy it back

- Course text: 4.17 bytes per token
- Merges trade vocabulary rows for length
- Full coverage needs all 256 bytes
- Monad keeps 207; course text unaffected
- Course tokenizer: Monad, 8,192 entries (PleIAs, 2025)

▼ The answer, and the course's choice

So why not feed the model raw bytes? Raw bytes cover every input, but they cost one position per byte. On the course's English training text, Monad averages 4.17 bytes per token. With byte input, the same text would need about four times as many positions, with the attention work that comes with them, and a context window of fixed length would hold about a quarter as much text. Byte-level BPE keeps the coverage of bytes, as long as all 256 byte symbols stay in the vocabulary, and spends vocabulary rows to buy shorter sequences. How many rows, which merges, and whether every byte symbol is kept are choices you measure on the target data.

Monad shows both sides on the comparison widget's sentences. Its 7,981 learned merges take the English sentence from 72 bytes to 10 tokens. The Japanese and Bengali sentences stay at one token per byte, because no Monad entry, whole or partial, covers their byte sequences. Monad also keeps only 207 of the 256 byte symbols, yet the course's English training text produced no `[UNK]` in about 20 million tokens.

The course reuses Monad with its 8,192 logical vocabulary entries (PleIAs, 2025). Reusing a tokenizer lets the course spend its time on the model, training and optimization, while you can still measure what segmentation does. Another language, domain, context budget or deployment target may call for a different tokenizer.

Back to the question